INFORME TÉCNICO DE ARQUITECTURA & SEGURIDAD

Hito #2: Integración Frontend HTML/CSS con Procesamiento Backend PHP

Asignatura: Desarrollo Web (Módulo III)

Modalidad: Equipo de 4 Integrantes (Rotación de Roles)

Repositorio: Semestral-Hito-1 (GitHub)

Puntaje Asignado: 40% de la Nota Final

1. DIAGRAMA Y FLUJO DE PETICIÓN CLIENTE/SERVIDOR

La transición de la maqueta estática del Hito #1 hacia una arquitectura dinámica en el Hito #2 introduce la ejecución de código en el servidor. Cada interacción del usuario desencadena un ciclo de vida síncrono donde el servidor HTTP interpreta la solicitud, procesa la lógica de negocio mediante scripts PHP aislados, y retorna un documento HTML saneado e interpretable por el navegador.

1.1 Secuencia de Ejecución de una Petición HTTP

1. **Emisión del Cliente (Navegador Web)**
El usuario interactúa con la interfaz (formulario POST o filtro GET en `principal_listado.php`). El navegador envía una petición HTTP al puerto 8000/80 de Apache/PHP Engine conteniendo cabeceras y cuerpo (payload).

2. **Intercepción por el Servidor Web y Control de Acceso**
El servidor web recibe la petición. Si se intenta acceder directamente a un fichero protegido en `includes/` o `clases/`, el servidor intercepta la regla `.htaccess` o el script evalúa la constante `ACCESO_QZ7`, terminando la ejecución mediante `exit('Acceso denegado')`.

3. **Inicialización e Inclusión de Entorno (PHP Engine)**
El script objetivo (ej. `procesar_registro.php`) verifica el método HTTP mediante `$_SERVER['REQUEST_METHOD']`. Carga las dependencias esenciales: `includes/funciones.php` (para sanitización) y `clases/Validador.php` mediante `require_once`.

4. **Sanitización, Instanciación Orientada a Objetos y Reglas de Negocio**
Se extraen y purifican las variables (`sanear_zq()`). Se instancian las clases de entidad (`Usuario`, `Producto`) y la clase `Validador`. Se ejecutan las validaciones del servidor y la regla de negocio asignada al equipo.

5. **Respuesta HTTP y Renderizado de Salida Segura**
Si existen errores, se redirige o reconstruye el formulario conservando los valores válidos y desplegando mensajes claros. Si la validación es exitosa, se genera la vista de confirmación pasando todos los textos por `htmlspecialchars()` para mitigar ataques XSS.

1.2 Criterios de Selección del Método HTTP (GET vs. POST)

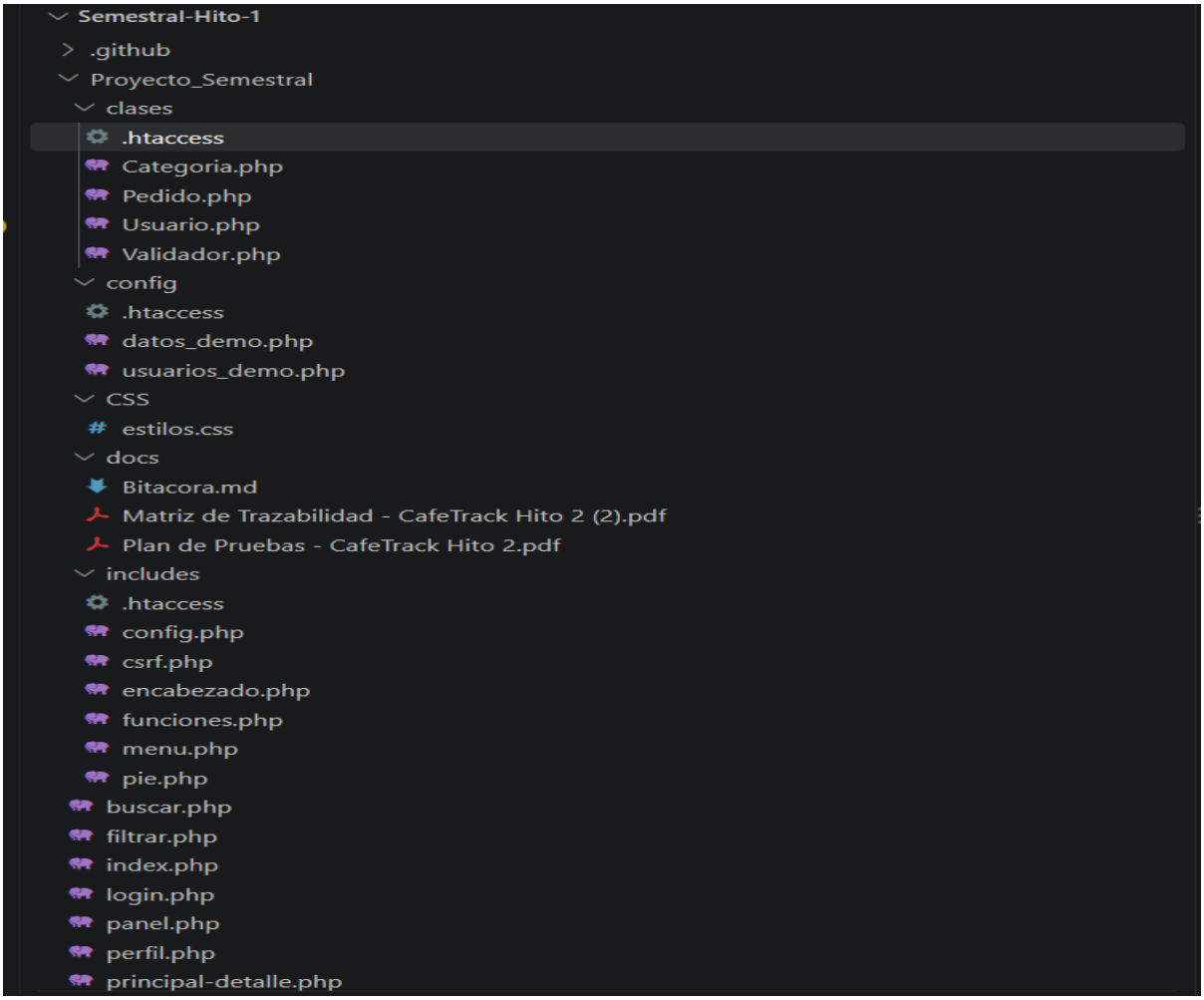
La arquitectura del sistema aplica de forma rigurosa los estándares HTTP especificados en el Temario 3.11, garantizando que el método utilizado responda a la naturaleza de la operación:

MÉTODO	CARACTERÍSTICAS TÉCHNICAS	USO EN EL SISTEMA	JUSTIFICACIÓN DE SEGURIDAD
GET	Idempotente, almacenable en caché, visibilidad de parámetros en URL, límite de longitud de cadena.	• Búsqueda general en `principal_listado.php` . • Filtros combinados(categoría/fecha). • Paginación (`$_GET['pagina']`).	Permite la navegabilidad y la creación de marcadores. Ninguna petición GET altera el estado del servidor ni modifica o inserta registros.
POST	No idempotente, datos sensibles en el cuerpo de la petición (payload), sin rastro en el historial ni en la URL.	• procesar_registro.php • procesar_login.php • Formulario Entidad Principal ySecundaria.	Evita la fuga de credenciales en logs de servidor o URLs. Es obligatorio para la modificación del estado del sistema y la creación de entidades.

2. ESTRUCTURA DE LA APLICACIÓN Y MODULARIZACIÓN

El proyecto ha sido estructurado siguiendo el principio de separación de responsabilidades, garantizando la mantenibilidad y modularidad del código fuente sin hacer uso de frameworks externos, cumpliendo con la arquitectura nativa exigida.

2.1 Jerarquía del Repositorio y Matriz de Componentes



2.2 Inclusión Modular y Control de Vistas por Rol

Las 10 páginas del sistema han sido totalmente convertidas a PHP, eliminando la duplicación de marcado mediante la instrucción require_once . El archivo includes/menu.php evalúa el estado del usuario simulado para ajustar la navegación en tiempo de ejecución.

Listados Generados Dinámicamente: Los listados de datos presentados en principal_listado.php no contienen filas escritas a mano. Se iteran mediante bucles foreach consumiendo el arreglo asociativo almacenado en config/datos_demo.php . Dicho archivo incluye rigurosamente los 3 registros dictados por el docente en la semana 9. Asimismo, la paginación segmenta el arreglo a 5 elementos por página utilizando la variable $_GET['pagina'] validada numéricamente.

3. TABLA DE VALIDACIONES EN EL SERVIDOR

Bajo la premisa fundamental de seguridad de que *"todo dato proveniente del cliente es malicioso"*, el backend ejecuta una validación exhaustiva en PHP. Quitar los atributos required , modificar elementos <select> mediante inspección DOM, o alterar parámetros en la URL es neutralizado en el servidor mediante listas blancas y verificaciones de tipo estricto.

3.1 Matriz General de Validación de Formularios

CAMPO / ENTRADA	FORMULARIO	REGLA APLICADA EN BACKEND (PHP)	MENSAJE DE ERROR PRESENTADO
nombre / apellido	Registro / Perfil	Obligatorio. Longitud entre 2 y 50 caracteres. Expresión regular /^[a-zAZáéíóúÁÉÍÓÚñÑ\s]+$/u .	"El nombre solo debe contener letras y tener entre 2 y 50 caracteres."
correo	Registro / Login	Obligatorio. Validación de formato RFC mediante filter_var($email, FILTER_VALIDATE_EMAIL) .	"Ingrese una dirección de correo electrónico válida."
contrasena	Registro	Obligatorio. Mínimo 8 caracteres, al menos una mayúscula, un número y un carácter especial.	"La contraseña debe tener al menos 8 caracteres, una mayúscula y un número."
categoria_id	Entidad Principal	Obligatorio. Lista blanca (*whitelisting*) validada con in_array($val, $categorias_permitidas) .	"La categoría seleccionada no es válida."
precio / monto	Entidad Principal	Obligatorio. Numérico flotante is_numeric() , valor mayor a 0 y menor a 10,000.	"El precio debe ser un número positivo válido."
fecha_alta	Entidad Secundaria	Formato YYYY-MM-DD comprobado con DateTime::createFromFormat() . Fecha no futura.	"La fecha ingresada no tiene un formato válido o es posterior al día de hoy."
campo_especial_qz (Personalizado)	Entidad Principal	Verificación del prefijo asignado (QZ-) seguido de 6 dígitos numéricos.	"El código de inventario debe iniciar con el prefijo 'QZ-' seguido de 6 dígitos."
CAMPO / ENTRADA	FORMULARIO	REGLA APLICADA EN BACKEND (PHP)	MENSAJE DE ERROR PRESENTADO

regla_negocio_monto (Regla Asignada)	Formulario General	Validación en servidor: Si la categoría es 'Premium', el monto mínimo no puede ser menor a $100.00.	"Los productos de categoría Premium requieren un monto mínimo inicial de $100.00."
pagina	Navegación GET	Conversión a entero (int) . Si es < 1 o excede el total de páginas, asigna página 1.	*Redirección silenciosa a página 1 (sin fallos PHP).*

3.2 Arquitectura del Motor de Validación (Clase POO Validador)

Para mantener el código DRY (*Don't Repeat Yourself*), el procesamiento de errores se abstrae en la clase Validador.php , la cual acumula las inconsistencias y permite retornar al formulario conservando los campos previamente rellenados que resultaron válidos.

4. DECISIONES DE SEGURIDAD WEB Y MITIGACIÓN DE VULNERABILIDADES

El diseño del backend aplica principios estrictos de seguridad defensiva alineados con el Temario del Módulo III. Cada vector de ataque potencial ha sido contrarrestado con una técnica específica de mitigación.

4.1 Matriz de Implementación de Seguridad

AMENAZA / VECTOR	TÉCNICA DE MITIGACIÓN	IMPLEMENTACIÓN CONCRETA EN EL CÓDIGO	TEMARIO
Cross-Site Scripting (XSS)	Escapado completo de salida HTML en toda variable dinámicamente renderizada.	Uso sistemático de la función de sanitización en funciones.php : htmlspecialchars($dato, ENT_QUOTES, "UTF-8") . Se verificó inyectando <script>alert(1)</script> , el cual se renderiza como texto plano.	Temario 3.12
Almacenamiento de Contraseñas	Cifrado unidireccional (Hashing algorítmico seguro).	Registro: password_hash($pass, PASSWORD_DEFAULT) . Autenticación: password_verify($pass, $hashGuardado) . *Jamás se almacena, transmite o imprime texto plano.*	Temario 3.13
Acceso Directo a Ficheros de Biblioteca	Aislamiento de scripts en includes/ y clases/ mediante constantes y .htaccess .	En páginas públicas: define('ACCESO_QZ7', true); . En cabecera de includes/clases: defined('ACCESO_QZ7') or exit('Acceso denegado'); . Fichero .htaccess con Require all denied .	Temario 3.14.7
AMENAZA / VECTOR	TÉCNICA DE MITIGACIÓN	IMPLEMENTACIÓN CONCRETA EN EL CÓDIGO	TEMARIO

Manipulación de Métodos HTTP	Restricción de métodos y desestimación de peticiones inválidas.	Validación explícita al inicio de scripts de procesamiento: `if ($_SERVER['REQUEST_METHOD'] !== 'POST') { header('Location: index.php'); exit; } .`	Temario 3.11
Fuga de Información Tecnológica	Manejo controlado de excepciones y mensajes de error genéricos.	Desactivación de desplegado de errores internos en producción (`display_errors = Off`). Los mensajes mostrados al usuario son informativos y no revelan rutas del servidor.	Temario 3.12
Bonus: Cross-Site Request Forgery (CSRF)	Tokens aleatorios de un solo uso por sesión para formularios POST.	Generación de token con `bin2hex(random_bytes(32))` almacenado en sesión e inyectado en campo oculto `<input type="hidden" name="csrf_token">` .	Temario 3.12

4.2 Demostración del Mecanismo de Protección contra Acceso Directo

Si un atacante intenta navegar directamente hacia la URL `http://localhost:8000/includes/funciones.php` o `http://localhost:8000/clases/Usuario.php` , el servidor detiene inmediatamente el hilo de ejecución:

5. ANÁLISIS DE DESPLIEGUE EN PRODUCCIÓN

Para llevar la aplicación actual desde el entorno de desarrollo local (XAMPP/Laragon) hacia un entorno de producción accesible en Internet, se analizaron dos opciones tecnológicas evaluando costos, complejidad y factibilidad.

5.1 Comparativa de Opciones de Alojamiento Web

CRITERIO	OPCIÓN A: SHARED HOSTING (CPANEL / HOSTINGER)	OPCIÓN B: CLOUD VPS (DIGITALOCEAN / LINODE)
Infraestructura	Servidor compartido preconfigurado con cPanel, Apache y PHP 8.x nativo.	Servidor Privado Virtual (Ubuntu 22.04 LTS) con pila Nginx/Apache + PHP-FPM.
Ventajas	Baja complejidad de administración, certificados SSL Let's Encrypt automatizados, gestión de BD mediante phpMyAdmin visual.	Control total de la máquina (acceso SSH), aislamiento absoluto de recursos, alta capacidad de escalamiento horizontal.
Desventajas	Recursos de CPU y RAM compartidos con otros sitios, menor flexibilidad para instalar librerías personalizadas.	Requiere conocimientos avanzados de administración de sistemas Linux, seguridad en red, parches del SO y firewall.
Costo Estimado	~$3.50 a $6.00 USD / mes (Facturado anualmente).	~$6.00 a $12.00 USD / mes (Según instancias RAM/CPU).
CRITERIO	**OPCIÓN A: SHARED HOSTING (CPANEL / HOSTINGER)**	**OPCIÓN B: CLOUD VPS (DIGITALOCEAN / LINODE)**

Recomendación	**Opción A seleccionada:** Para el alcance de este proyecto semestral, un Hosting Compartido ofrece un entorno PHP 8.x con MySQL totalmente optimizado, reduciendo costos operativos y tiempo de mantenimiento.

6. REFLEXIONES INDIVIDUALES DE LOS INTEGRANTES

A continuación se presentan las reflexiones técnicas personales sobre los problemas resueltos y los aprendizajes adquiridos durante la ejecución del Hito #2, en función de los roles rotados:

Integrante 1: Sergio Ceballos

LÍDER DE PROYECTO Y REPOSITORIO

"Durante este hito asumí el liderazgo del repositorio de Git y la gestión de la integración. Mi mayor reto técnico fue resolver conflictos de fusión (*merge conflicts*) cuando dos compañeros modificaron simultáneamente la inclusión de dependencias en `index.php`. Para solucionarlo, establecí una regla estricta de Pull Requests donde nadie fusionaba su propio código sin la aprobación del revisor asignado. Aprendí la importancia crucial de mantener ramas de características aisladas (`rama/funcionalidad`) y a verificar que el historial de Git reflejara aportes distribuidos equitativamente sin commits masivos."

Integrante 2: Francisco Quiñones

DISEÑO Y EXPERIENCIA DE USUARIO (UX/UI)

"Mi responsabilidad principal fue migrar las 10 maquetas estáticas a plantillas dinámicas `.php` y construir el menú por rol. Al principio, la navegación se rompía porque los arreglos de sesión no conservaban el estado del usuario simulado entre páginas. Lo resolví unificando la inclusión del encabezado y el inicio de sesión mediante `session_start()` en `includes/encabezado.php`. Aprendí a estructurar la paginación del listado principal consumiendo variables GET y a mantener una interfaz limpia conservando los valores válidos introducidos por el usuario cuando el servidor devuelve un error de validación."

Integrante 3: Eric ALveo

DESARROLLO BACKEND PHP

"Como encargado del desarrollo backend, diseñé la arquitectura orientada a objetos (las 4 clases en `clases/`) y las funciones de `includes/funciones.php`. Un problema complejo fue lograr que la clase `Usuario` verificara correctamente los hashes de contraseñas. Inicialmente comparaba directamente la contraseña introducida con la cadena hash usando operadores de igualdad, lo que provocaba fallos lógicos. Tras revisar la documentación de PHP, implementé `password_verify()` y abstraje los métodos getters y setters con validación interna. Este proceso me enseñó la potencia de la encapsulación en POO para proteger la lógica de negocio."

Integrante 4: Omar Mojica

CALIDAD, SEGURIDAD Y DOCUMENTACIÓN

"Lideré la validación de seguridad y la elaboración del plan de pruebas. El mayor desafío fue garantizar que ningún archivo de biblioteca pudiera ejecutarse mediante acceso directo en el navegador. Al probar las URLs directamente, descubrí que algunos archivos en `includes/` aún mostraban advertencias PHP. Lo corregí implementando la constante

ACCESO_QZ7 en todas las cabeceras e integrando la sanitización con htmlspecialchars() en las salidas. Aprendí a pensar como un auditor de seguridad: nunca confiar
en las validaciones de HTML del cliente y verificar minuciosamente cada vector de entrada."